

TASK 2

Application Development Security

APDS7311

ST10099624

GOMOLEMO MOTLHAMME

GITHUB: <https://github.com/GMotlhamme/ClarityCapitalBankInc>

YOUTUBE: <https://youtu.be/pwSN2FU0-Xw>

APPLICATION ANALYSIS

This project uses the ASP.NET Core Web API to provide a safe international payment system. Customers can register, authenticate, and conduct international transactions through the system, and staff members validate and handle these transactions. Several security safeguards were put in place in accordance with secure software development principles since financial and personal data is important (OWASP, 2023).

The framework was used to create the application's RESTful API architecture. Microsoft Identity is used for user administration and authentication, and Entity Framework Core is used for database interaction (Microsoft, 2024). Users (Employees and Customers) and payments are the two primary components of the system.

JSON Web Tokens (JWT) are used to implement authentication. A token with user claims such as email address, username, and role is created when a user checks in. Access to protected endpoints requires this token (Jones et al., 2015). Role-based authorization assigns users to roles such as "Customer" or "Employee." These roles use the [Authorize(Roles = "the role")] attribute to restrict access to certain endpoints (Microsoft, 2024).

BCrypt hashing is used to protect passwords. Passwords are never kept in plain text thanks to this. To prevent brute-force and rainbow table attacks, BCrypt automatically creates a salt and uses a computationally costly hashing algorithm (OWASP, 2023).

Every user's input is regarded as untrusted. User input is securely received through the use of Data Transfer Objects (DTOs). Regular expressions and data annotations are used to enforce validation and guarantee proper formats (such as currency codes and SWIFT codes). Before interacting with the database, ModelState validation makes sure that only valid and appropriately structured data is processed (OWASP, 2023)

Entity Framework Core prevents SQL Injection by using parameterized queries by default. By doing this, user input is prevented from being directly processed as SQL statements (OWASP, 2023). Regarding Securing Data in Transit the application enforces HTTPS using `app.UseHttpsRedirection()` reinforced by HTTP Strict Transport Security (HSTS). This guarantees that all client-server communication is encrypted, guarding against man-in-the-middle attacks and interception (IETF, 2018; OWASP, 2023).

Cross-Site Scripting (XSS) is prevented on the frontend using React, which automatically escapes user input when rendering content. This stops the browser from running dangerous scripts (Meta, 2024). Additionally, backend validation lessens the possibility of processing fraudulent data (OWASP, 2023).

Clickjacking is prevented by setting security headers in middleware, specifically the Content Security Policy (CSP) with frame-ancestors 'none'. This stops iframes from embedding the application (MDN Web Docs, 2024). For Session Security JWT tokens are used in place of conventional sessions. These tokens lower the danger of session hijacking because they are short-lived and must be included in every request (Jones et al., 2015; OWASP, 2023).

Rate limiting is used by the program to limit how many requests a user may submit in a specific amount of time. This guarantees service availability and lessens the impact of Distributed Denial-of-Service (DDoS) attacks (NIST, 2007).

GitHub Actions was used to create a continuous integration pipeline. The pipeline supports safe and dependable software distribution methods by automatically building the application and validating code modifications (GitHub, 2024).

CONCLUSION

By integrating several security layers, including authentication, input validation, transport security, and enterprise-level protections. The system demonstrates a defense-in-depth strategy. This guarantees critical financial data's availability, confidentiality, and integrity (OWASP, 2023).

REFERENCE LIST

GitHub (2024) GitHub Actions Documentation. Available at: <https://docs.github.com> [Accessed: 4 May 2026]

IETF (2018) RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. Available at: <https://datatracker.ietf.org/doc/html/rfc8446> [Accessed: 4 May 2026]

Jones, M., Bradley, J. and Sakimura, N. (2015) JSON Web Token (JWT) RFC 7519. Available at: <https://datatracker.ietf.org/doc/html/rfc7519> [Accessed: 4 May 2026]

MDN Web Docs (2024) Content Security Policy (CSP). Available at: <https://developer.mozilla.org> [Accessed: 4 May 2026]

Meta (2024) React Documentation. Available at: <https://react.dev> (Accessed: 4 May 2026).

Microsoft (2024) ASP.NET Core Documentation. Available at: <https://learn.microsoft.com> [Accessed: 4 May 2026]

NIST (2007) Guide to Intrusion Detection and Prevention Systems (IDPS). Available at: <https://nvlpubs.nist.gov> [Accessed: 4 May 2026]

OWASP (2023) OWASP Top Ten and Security Cheat Sheets. Available at: <https://owasp.org> [Accessed: 4 May 2026]

